

Experiment 02:

Laboratory Exercise

Part A: Identifying and performing data cleaning operations

1. Missing values

a. *dropna()*

Code:

```
print(df2)
print("\n-----dropna()-----\n")
```

```
rowsRemovedDataFrame = df2.dropna()
```

```
print(rowsRemovedDataFrame)
```

Output:

```
...      sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2     setosa
1           4.9           3.0           1.4           0.2     setosa
2           NaN           3.2           1.3           0.2     setosa
3           4.6           3.1           1.5           0.2     setosa
4           5.0           3.6           1.4           NaN     setosa
..          ...           ...           ...           ...           ...
148          6.2           3.4           5.4           2.3  virginica
149          5.9           3.0           5.1           1.8  virginica
150          NaN           3.2           1.3           0.2     setosa
151          NaN           3.2           1.3           0.2     setosa
152          NaN           3.2           1.3           0.2     setosa

[153 rows x 5 columns]

-----dropna()-----

      sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2     setosa
1           4.9           3.0           1.4           0.2     setosa
3           4.6           3.1           1.5           0.2     setosa
5           5.4           3.9           1.7           0.4     setosa
6           4.6           3.4           1.4           0.3     setosa
..          ...           ...           ...           ...           ...
145          6.7           3.0           5.2           2.3  virginica
146          6.3           2.5           5.0           1.9  virginica
147          6.5           3.0           5.2           2.0  virginica
148          6.2           3.4           5.4           2.3  virginica
149          5.9           3.0           5.1           1.8  virginica

[148 rows x 5 columns]
```

b. dropna(axis=1)**Code:**

```
print("-----dropna(axis=1)-----")
```

```
columnsRemovedDataFrame = df2.dropna(axis=1)
```

```
print(columnsRemovedDataFrame)
```

Output:

```
... -----dropna(axis=1)-----
   sepal_width  petal_length  species
0           3.5           1.4   setosa
1           3.0           1.4   setosa
2           3.2           1.3   setosa
3           3.1           1.5   setosa
4           3.6           1.4   setosa
..          ...           ...     ...
148          3.4           5.4  virginica
149          3.0           5.1  virginica
150          3.2           1.3   setosa
151          3.2           1.3   setosa
152          3.2           1.3   setosa

[153 rows x 3 columns]
```

c. dropna(how='all')**Code:**

```
print(df2)
```

```
print("\n-----dropna(how='all')-----\n")
```

```
allMissingRowsRemovedDataFrame = df2.dropna(how="all")
```

```
print(allMissingRowsRemovedDataFrame)
```

Output:

```

...      sepal_length  sepal_width  petal_length  petal_width  species
0         5.1         3.5         1.4         0.2      setosa
1         4.9         3.0         1.4         0.2      setosa
2         NaN         3.2         1.3         0.2      setosa
3         4.6         3.1         1.5         0.2      setosa
4         5.0         3.6         1.4         NaN      setosa
..      ...      ...      ...      ...      ...
148        6.2         3.4         5.4         2.3  virginica
149        NaN         NaN         NaN         NaN         NaN
150        NaN         3.2         1.3         0.2      setosa
151        NaN         3.2         1.3         0.2      setosa
152        NaN         3.2         1.3         0.2      setosa

```

[153 rows x 5 columns]

-----dropna(how='all')-----

```

      sepal_length  sepal_width  petal_length  petal_width  species
0         5.1         3.5         1.4         0.2      setosa
1         4.9         3.0         1.4         0.2      setosa
2         NaN         3.2         1.3         0.2      setosa
3         4.6         3.1         1.5         0.2      setosa
4         5.0         3.6         1.4         NaN      setosa
..      ...      ...      ...      ...      ...
147        6.5         3.0         5.2         2.0  virginica
148        6.2         3.4         5.4         2.3  virginica
150        NaN         3.2         1.3         0.2      setosa
151        NaN         3.2         1.3         0.2      setosa
152        NaN         3.2         1.3         0.2      setosa

```

d. fillna()**Code:**

```

print("-----fillna()-----")

filledDataFrame = df2.copy()

filledDataFrame["sepal_length"] = filledDataFrame["sepal_length"].fillna(
    filledDataFrame["sepal_length"].mean()
)

filledDataFrame["petal_width"] = filledDataFrame["petal_width"].fillna(
    filledDataFrame["petal_width"].mean()
)

print(filledDataFrame)

```

Output:

```

... -----fillna()-----
      sepal_length  sepal_width  petal_length  petal_width  species
0         5.100000         3.5         1.4         0.200000    setosa
1         4.900000         3.0         1.4         0.200000    setosa
2         5.850676         3.2         1.3         0.200000    setosa
3         4.600000         3.1         1.5         0.200000    setosa
4         5.000000         3.6         1.4         1.181457    setosa
..         ...         ...         ...         ...         ...
148        6.200000         3.4         5.4         2.300000  virginica
149        5.850676         NaN         NaN         1.181457         NaN
150        5.850676         3.2         1.3         0.200000    setosa
151        5.850676         3.2         1.3         0.200000    setosa
152        5.850676         3.2         1.3         0.200000    setosa

[153 rows x 5 columns]

```

2. Duplicate records**a. drop_duplicates()****Code:**

```

print("-----drop_duplicates()-----\n")

removedDuplicatesDataFrame = df2.drop_duplicates()

print(removedDuplicatesDataFrame)

```

Output:

```

... -----drop_duplicates()-----
      sepal_length  sepal_width  petal_length  petal_width  species
0         5.1         3.5         1.4         0.2    setosa
1         4.9         3.0         1.4         0.2    setosa
2         NaN         3.2         1.3         0.2    setosa
3         4.6         3.1         1.5         0.2    setosa
4         5.0         3.6         1.4         NaN    setosa
..         ...         ...         ...         ...         ...
145        6.7         3.0         5.2         2.3  virginica
146        6.3         2.5         5.0         1.9  virginica
147        6.5         3.0         5.2         2.0  virginica
148        6.2         3.4         5.4         2.3  virginica
149         NaN         NaN         NaN         NaN         NaN

[147 rows x 5 columns]

```

b. *drop_duplicates(keep='last')***Code:**

```
print("-----drop_duplicates(keep='last')-----\n")

keepLastDuplicateDataFrame = df2.drop_duplicates(keep="last")

print(keepLastDuplicateDataFrame)
```

Output:

```
... -----drop_duplicates(keep='last')-----

   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2    setosa
1           4.9           3.0           1.4           0.2    setosa
3           4.6           3.1           1.5           0.2    setosa
4           5.0           3.6           1.4           NaN    setosa
5           5.4           3.9           1.7           0.4    setosa
..          ...           ...           ...           ...     ...
146          6.3           2.5           5.0           1.9  virginica
147          6.5           3.0           5.2           2.0  virginica
148          6.2           3.4           5.4           2.3  virginica
149          NaN           NaN           NaN           NaN      NaN
152          NaN           3.2           1.3           0.2    setosa

[147 rows x 5 columns]
```

c. *df.duplicated()***Code:**

```
print("-----duplicated()-----")
duplicateRows = df2.duplicated()
print(duplicateRows)
```

Output:

```
... -----duplicated()-----

0    False
1    False
2    False
3    False
4    False
...
148  False
149  False
150   True
151   True
152   True
Length: 153, dtype: bool
```

3. Incorrect data types, changing data types

a. astype()

Code:

```
print("-----astype()-----\n")

# Converting the target column to integer type
df["sepal_length"] = df["sepal_length"].astype(int)

print(df.dtypes)
print(df.head())
```

Output:

```
... -----astype()-----

sepal_length    int64
sepal_width     float64
petal_length     float64
petal_width     float64
species         object
dtype: object
   sepal_length  sepal_width  petal_length  petal_width  species
0             5           3.5           1.4           0.2   setosa
1             4           3.0           1.4           0.2   setosa
2             4           3.2           1.3           0.2   setosa
3             4           3.1           1.5           0.2   setosa
4             5           3.6           1.4           0.2   setosa
```

4. Drop irrelevant col

a. drop(columns=[])

Code:

```
print("-----drop(columns=[])------\n")
updatedDataFrame = df.drop(columns=["petal_width"])

print(updatedDataFrame.head())
```

Output:

```
... -----drop(columns=[])------

   sepal_length  sepal_width  petal_length  species
0             5.1           3.5           1.4   setosa
1             4.9           3.0           1.4   setosa
2             4.7           3.2           1.3   setosa
3             4.6           3.1           1.5   setosa
4             5.0           3.6           1.4   setosa
```

5. Drop irrelevant row

b. dropna(subset=[])

Code:

```
print("-----dropna(subset=[])-----")

cleanedSubsetDataFrame = df2.dropna(subset=["sepal_length"])

print(cleanedSubsetDataFrame)
```

Output:

```
... -----dropna(subset=[])-----
      sepal_length  sepal_width  petal_length  petal_width  species
0             5.1           3.5           1.4           0.2    setosa
1             4.9           3.0           1.4           0.2    setosa
3             4.6           3.1           1.5           0.2    setosa
4             5.0           3.6           1.4           NaN    setosa
5             5.4           3.9           1.7           0.4    setosa
..            ...           ...           ...           ...      ...
144            6.7           3.3           5.7           2.5  virginica
145            6.7           3.0           5.2           2.3  virginica
146            6.3           2.5           5.0           1.9  virginica
147            6.5           3.0           5.2           2.0  virginica
148            6.2           3.4           5.4           2.3  virginica

[148 rows x 5 columns]
```

6. Spelling and typographical errors

a. str.lower()

Code:

```
print("-----str.lower()-----")

df["Name"] = df["Name"].str.lower()

print(df)
```

Output:

```
... -----str.lower()-----
      Sr  Age  Name  ID
0     1   20  radha  101
1     2   21  shivam  102
2     3   19 ashish36  136
3     4   22   mansi  163
4     5   20   nitin  164
```

b. str.upper()**Code:**

```
print("-----str.upper()-----")
```

```
df["Name"] = df["Name"].str.upper()
```

```
print(df)
```

Output:

```
... -----str.upper()-----
   Sr  Age   Name  ID
0   1   20  RADHA  101
1   2   21  SHIVAM  102
2   3   19  ASHISH36  136
3   4   22   MANSI  163
4   5   20   NITIN  164
```

c. str.title()**Code:**

```
print("-----str.title()-----")
```

```
df["Name"] = df["Name"].str.title()
```

```
print(df)
```

Output:

```
... -----str.title()-----
   Sr  Age   Name  ID
0   1   20  Radha  101
1   2   21  Shivam  102
2   3   19  Ashish36  136
3   4   22   Mansi  163
4   5   20   Nitin  164
```

d. str.strip()**Code:**

```
print("-----str.strip()-----")
```

```
df["Name"] = df["Name"].str.strip()
```

```
print(df)
```


Output:

```
-----str.strip()-----
   Sr  Age   Name  ID
0   1   20   Radha 101
1   2   21  Shivam 102
2   3   19 Ashish36 136
3   4   22   Mansi 163
4   5   20   Nitin 164
```

e. str.replace()**Code:**

```
print("-----str.replace()-----")
```

```
df["Name"] = df["Name"].str.replace(r"\d+", "", regex=True)
```

```
print(df)
```

Output:

```
... -----str.replace()-----
   Sr  Age   Name  ID
0   1   20   Radha 101
1   2   21  Shivam 102
2   3   19  Ashish 136
3   4   22   Mansi 163
4   5   20   Nitin 164
```

f. unique()**Code:**

```
print("-----unique()-----\n")
```

```
print(df["Name"].unique())
```

Output:

```
... -----unique()-----

['Radha' 'Shivam' 'Ashish' 'Mansi' 'Nitin']
```

Part B: Applying data transformation techniques

B.1 Outliers and their detection methods

1. IQR Method

Code:

```
#Step 1 : Load the Dataset
```

```
iris = pd.read_csv("/kaggle/input/iris-dataset/iris.csv")
df1 = pd.DataFrame(iris)

print("\n-----HEAD-----\n")
print(df1.head())
```

Output:

```
...
-----HEAD-----
   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2   setosa
1           4.9           3.0           1.4           0.2   setosa
2           4.7           3.2           1.3           0.2   setosa
3           4.6           3.1           1.5           0.2   setosa
4           5.0           3.6           1.4           0.2   setosa
```

Code:

```
#Step 2 : Create Outliers and Select the Feature
```

```
# Creating Outliers
```

```
df1.loc[2, "sepal_length"] = 9
df1.loc[4, "sepal_length"] = 10
```

```
# Selecting the Feature
feature = df1["sepal_length"]
```

```
print(feature)
```

Output:

```
... 0      5.1
     1      4.9
     2      9.0
     3      4.6
     4     10.0
     ...
    145      6.7
    146      6.3
    147      6.5
    148      6.2
    149      5.9
    Name: sepal_length, Length: 150, dtype: float64
```

Code:

```
#Step 3 : Calculate Quartiles
```

```
Q1 = feature.quantile(0.25)
```

```
Q2 = feature.quantile(0.50)
```

```
Q3 = feature.quantile(0.75)
```

```
print("Q1 :", Q1)
```

```
print("Q2 :", Q2)
```

```
print("Q3 :", Q3)
```

Output:

```
... Q1 : 5.1
    Q2 : 5.8
    Q3 : 6.4
```

Code:

```
#Step 4 : Calculate Inter Quartile Range (IQR)
```

```
IQR = Q3 - Q1
```

```
print("IQR :", IQR)
```

Output:

```
... IQR : 1.3000000000000007
```

Code:

```
#Step 5 : Calculate Lower and Upper Bounds
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
print("Lower Bound :", lower)
```

```
print("Upper Bound :", upper)
```

Output:

```
... Lower Bound : 3.1499999999999986
    Upper Bound : 8.350000000000001
```

Code:

```
#Step 6 : Detect Outliers
```

```
outliers = df1[(feature < lower) | (feature > upper)]
```

```
print("Detected Outliers")
```

```
print(outliers)
```

Output:

```
... Detected Outliers
   sepal_length  sepal_width  petal_length  petal_width species
2           9.0           3.2           1.3           0.2  setosa
4          10.0           3.6           1.4           0.2  setosa
```

Code:

```
#Step 7 : Plot the Box Plot
```

```
plt.figure(figsize=(6,4))
```

```
plt.boxplot(feature, vert=True)
```

```
# Draw Horizontal Lines for Quartiles
```

```
plt.axhline(Q1, color="blue", linestyle="--", label="Q1")
```

```
plt.axhline(Q2, color="green", linestyle="--", label="Median (Q2)")
```

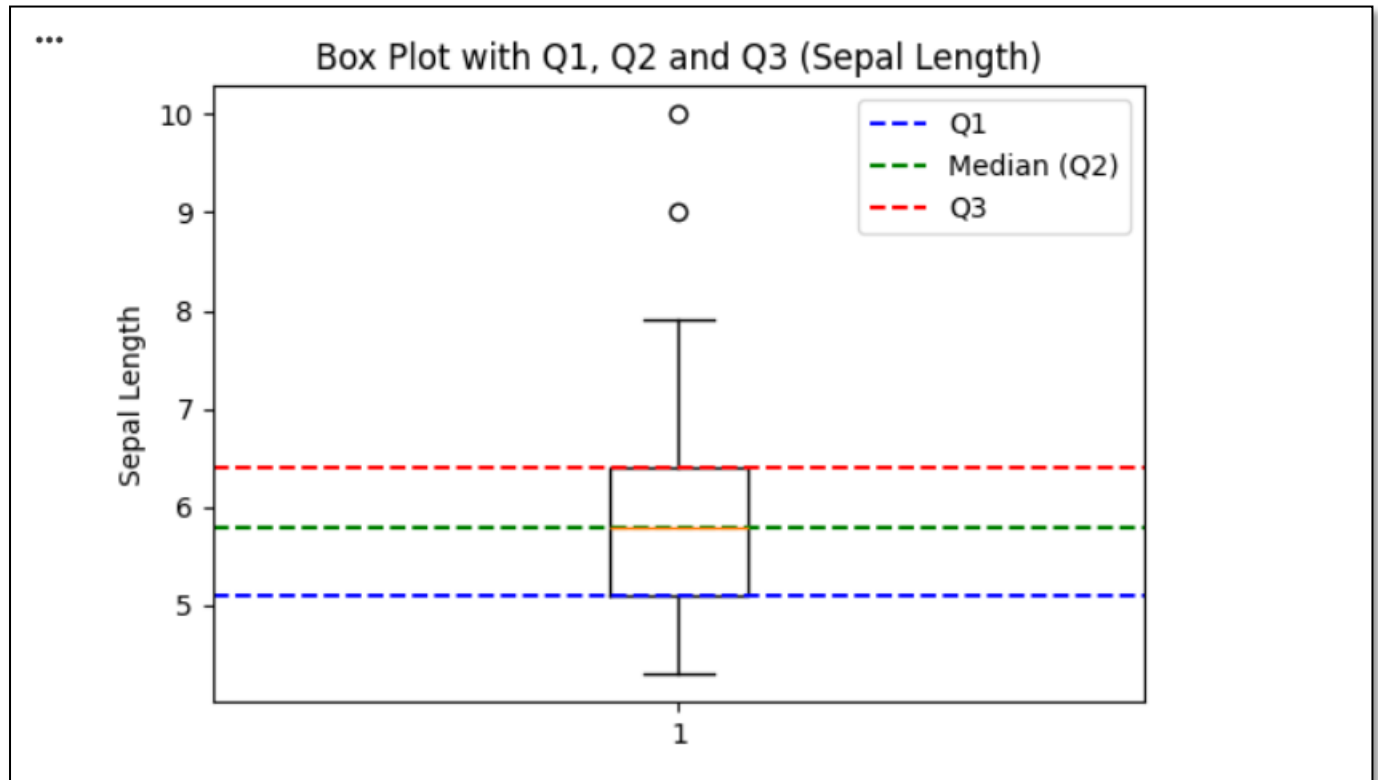
```
plt.axhline(Q3, color="red", linestyle="--", label="Q3")
```

```
plt.title("Box Plot with Q1, Q2 and Q3 (Sepal Length)")
```

```
plt.ylabel("Sepal Length")
```

```
plt.legend()
```

```
plt.show()
```

Output:**2. Z-Score Method**

Code:

#Step 1 : Load the Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

```
iris = pd.read_csv("/kaggle/input/iris-dataset/iris.csv")
```

```
df1 = pd.DataFrame(iris)
```

```
print(df1.head())
```

Output:

...	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Code:

```
#Step 2 : Select the Feature
```

```
feature = df1["sepal_length"]  
print(feature)
```

Output:

```
... 0      5.1  
    1      4.9  
    2      4.7  
    3      4.6  
    4      5.0  
    ...  
   145     6.7  
   146     6.3  
   147     6.5  
   148     6.2  
   149     5.9  
    Name: sepal_length, Length: 150, dtype: float64
```

Code:

```
#Step 3 : Calculate Mean and Standard Deviation
```

```
mean = feature.mean()  
standardDeviation = feature.std()  
  
print("Mean :", mean)  
print("Standard Deviation :", standardDeviation)
```

Output:

```
... Mean : 5.843333333333334  
    Standard Deviation : 0.8280661279778629
```

Code:

```
#Step 4 : Generate the Gaussian Distribution
```

```
x = np.linspace(  
    feature.min() - 3 * standardDeviation,  
    feature.max() + 3 * standardDeviation,1000)  
  
y = norm.pdf(x, mean, standardDeviation)
```

Code:

#Step 5 : Plot the Normal Distribution Curve

```
plt.figure(figsize=(12,6))
plt.plot(x, y, color="black", linewidth=3, label="Normal Distribution")
```

#Step 6 : Mark the Z-Score Regions

Mean

```
plt.axvline(mean,color="black",linestyle="--",label="Mean")
```

$\pm 1\sigma$

```
plt.axvline(mean-standardDeviation,color="green",linestyle="--")
plt.axvline(mean+standardDeviation,color="green",linestyle="--")
```

$\pm 2\sigma$

```
plt.axvline(mean-2*standardDeviation,color="orange",linestyle="--")
plt.axvline(mean+2*standardDeviation,color="orange",linestyle="--")
```

$\pm 3\sigma$

```
plt.axvline(mean-3*standardDeviation,color="red",linestyle="--")
plt.axvline(mean+3*standardDeviation,color="red",linestyle="--")
```

```
plt.fill_between(
    x,y,
    where=(x>=mean-standardDeviation)&(x<=mean+standardDeviation),
    color="yellow",
    alpha=0.6,
    label="Normal Region"
)
```

```
plt.fill_between(
    x,y,
    where=((x>=mean-2*standardDeviation)&(x<mean-standardDeviation))|
          ((x>mean+standardDeviation)&(x<=mean+2*standardDeviation)),
    color="lightgreen",
    alpha=0.6,
    label="Slightly Unusual"
)
```

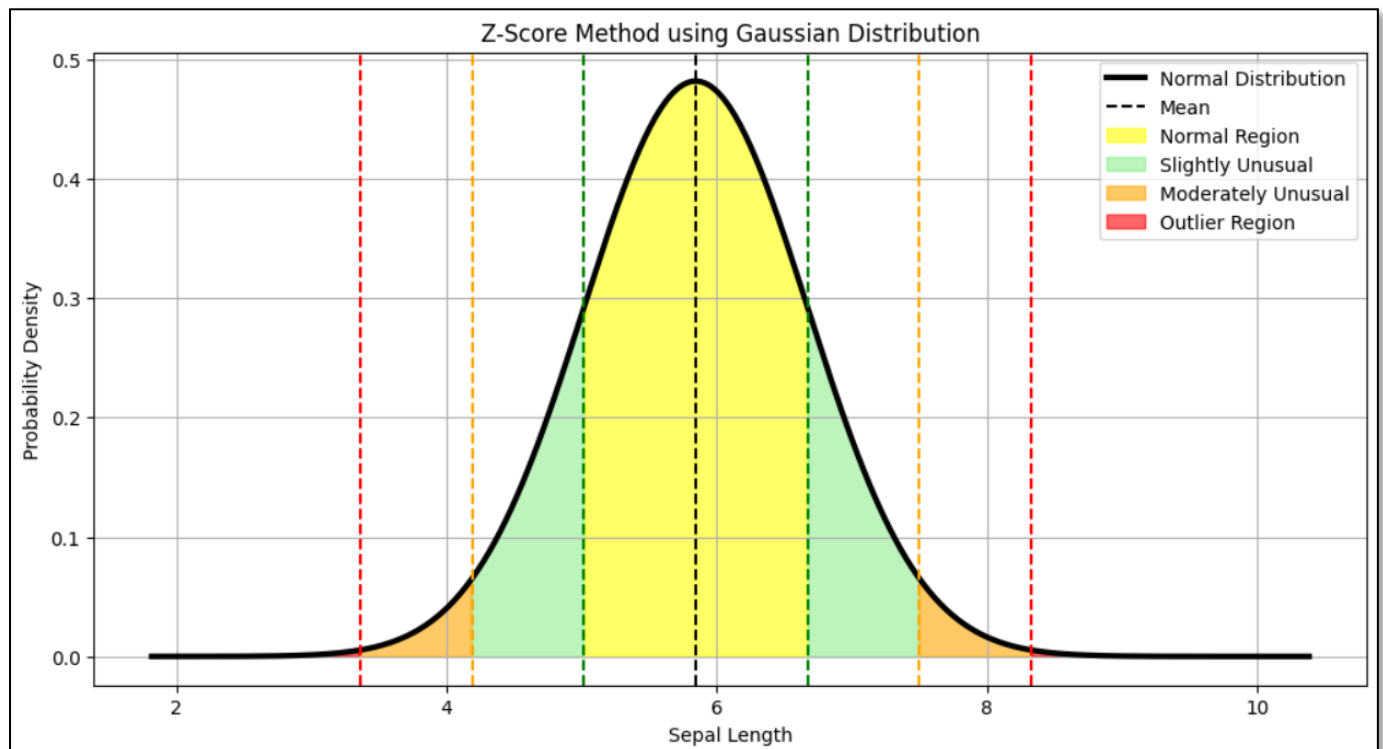
```
plt.fill_between(
    x,y,
    where=((x>=mean-3*standardDeviation)&(x<mean-2*standardDeviation))|
          ((x>mean+2*standardDeviation)&(x<=mean+3*standardDeviation)),
    color="orange",
)
```

```
    alpha=0.6,  
    label="Moderately Unusual"  
)  
  
plt.fill_between(  
    x,y,  
    where=(x<mean-3*standardDeviation)|(x>mean+3*standardDeviation),  
    color="red",  
    alpha=0.6,  
    label="Outlier Region"  
)
```

#Step 7 : Display the Graph

```
plt.title("Z-Score Method using Gaussian Distribution")  
plt.xlabel("Sepal Length")  
plt.ylabel("Probability Density")  
plt.legend()  
plt.grid(True)  
plt.show()
```

Output:



Code:

#Step 8 : Apply Standardization (Z-Score Scaling)

```
from sklearn.preprocessing import StandardScaler
```

```
# Creating Sample Outliers
```

```
df1.loc[2, "sepal_length"] = 12
```

```
df1.loc[4, "sepal_length"] = 13
```

```
# Selecting the Feature
```

```
feature = df1[["sepal_length"]]
```

```
# Applying Standardization
```

```
scaler = StandardScaler()
```

```
scaledFeature = scaler.fit_transform(feature)
```

```
scaledDataFrame = pd.DataFrame(  
    scaledFeature,  
    columns=["Scaled Sepal Length"]  
)
```

```
print(scaledDataFrame.head())
```

Output:

...	Scaled Sepal Length
0	-0.755663
1	-0.934447
2	5.412403
3	-1.202624
4	6.306326

Code:

#Step 9 : Plot Histograms Before and After Standardization

```
fig, axis = plt.subplots(1, 2, figsize=(14,5))
```

```
# Before Standardization
```

```
axis[0].hist(  
    feature["sepal_length"],  
    bins=12,  
    color="skyblue",  
    edgecolor="black"  
)
```

```
axis[0].set_title("Before Standardization")
```

```
axis[0].set_xlabel("Sepal Length")
```

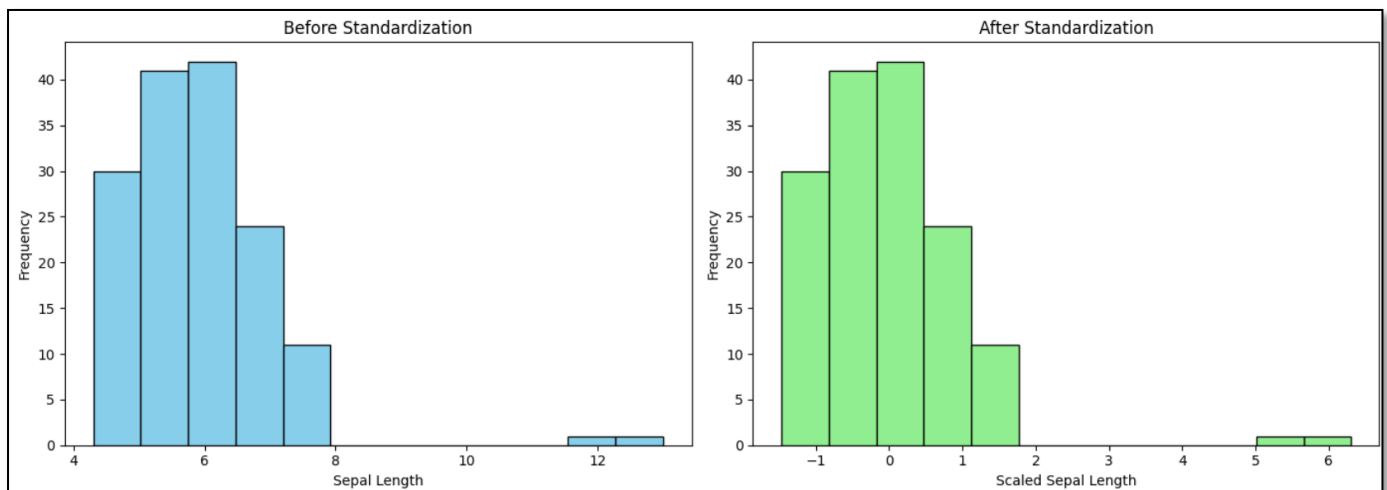
```
axis[0].set_ylabel("Frequency")

# After Standardization
axis[1].hist(
    scaledDataFrame["Scaled Sepal Length"],
    bins=12,
    color="lightgreen",
    edgecolor="black"
)

axis[1].set_title("After Standardization")
axis[1].set_xlabel("Scaled Sepal Length")
axis[1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Output:



B.2 Data Transformation Techniques

1. Absolute Maximum Scaling

Code:

#Step 1 : Select the Feature

```
feature = df1[["sepal_length"]]
print(feature.head())
```

Output:

```
...    sepal_length
0         5.1
1         4.9
2        12.0
3         4.6
4        13.0
```

Code:

#Step 2 : Apply Absolute Maximum Scaling

```
absoluteMaximumScaled = feature / feature.abs().max()
```

```
print(absoluteMaximumScaled.head())
```

Output:

```
...    sepal_length
0    0.392308
1    0.376923
2    0.923077
3    0.353846
4    1.000000
```

Code:

#Step 3 : Compare Original and Absolute Maximum Scaled Data

```
comparisonDataFrame = pd.DataFrame({
    "Original Sepal Length": feature["sepal_length"],
    "Absolute Maximum Scaled": absoluteMaximumScaled["sepal_length"]
})
```

```
print(comparisonDataFrame.head(10))
```

Output:

```
...    Original Sepal Length  Absolute Maximum Scaled
0         5.1                0.392308
1         4.9                0.376923
2        12.0                0.923077
3         4.6                0.353846
4        13.0                1.000000
5         5.4                0.415385
6         4.6                0.353846
7         5.0                0.384615
8         4.4                0.338462
9         4.9                0.376923
```

2. Min-Max Scaling

Code:

#Step 1 : Apply Min-Max Scaling

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
minMaxScaled = scaler.fit_transform(feature)
```

```
minMaxScaledDataFrame = pd.DataFrame(  
    minMaxScaled,  
    columns=["Min-Max Scaled"]  
)
```

```
print(minMaxScaledDataFrame.head())
```

Output:

```
...      Min-Max Scaled  
0      0.091954  
1      0.068966  
2      0.885057  
3      0.034483  
4      1.000000
```

Code:

#Step 2 : Compare Original and Min-Max Scaled Data

```
comparisonDataFrame = pd.DataFrame(  
    "Original Sepal Length": feature["sepal_length"],  
    "Min-Max Scaled": minMaxScaledDataFrame["Min-Max Scaled"]  
)  
print(comparisonDataFrame.head(10))
```

Output:

```
      Original Sepal Length  Min-Max Scaled  
0           5.1      0.091954  
1           4.9      0.068966  
2          12.0      0.885057  
3           4.6      0.034483  
4          13.0      1.000000  
5           5.4      0.126437  
6           4.6      0.034483  
7           5.0      0.080460  
8           4.4      0.011494  
9           4.9      0.068966
```

3. Standardization

Code:

#Step 1 : Apply Standardization

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

standardizedFeature = scaler.fit_transform(feature)

standardizedDataFrame = pd.DataFrame(
    standardizedFeature,
    columns=["Standardized Sepal Length"]
)
print(standardizedDataFrame.head())
```

Output:

...	Standardized Sepal Length
0	-0.755663
1	-0.934447
2	5.412403
3	-1.202624
4	6.306326

Code:

#Step 2 : Compare Original and Standardized Data

```
comparisonDataFrame = pd.DataFrame({
    "Original Sepal Length": feature["sepal_length"],
    "Standardized Sepal Length": standardizedDataFrame["Standardized Sepal Length"]
})
print(comparisonDataFrame.head(10))
```

Output:

	Original Sepal Length	Standardized Sepal Length
0	5.1	-0.755663
1	4.9	-0.934447
2	12.0	5.412403
3	4.6	-1.202624
4	13.0	6.306326
5	5.4	-0.487486
6	4.6	-1.202624
7	5.0	-0.845055
8	4.4	-1.381408
9	4.9	-0.934447

Post Experimental Exercise:

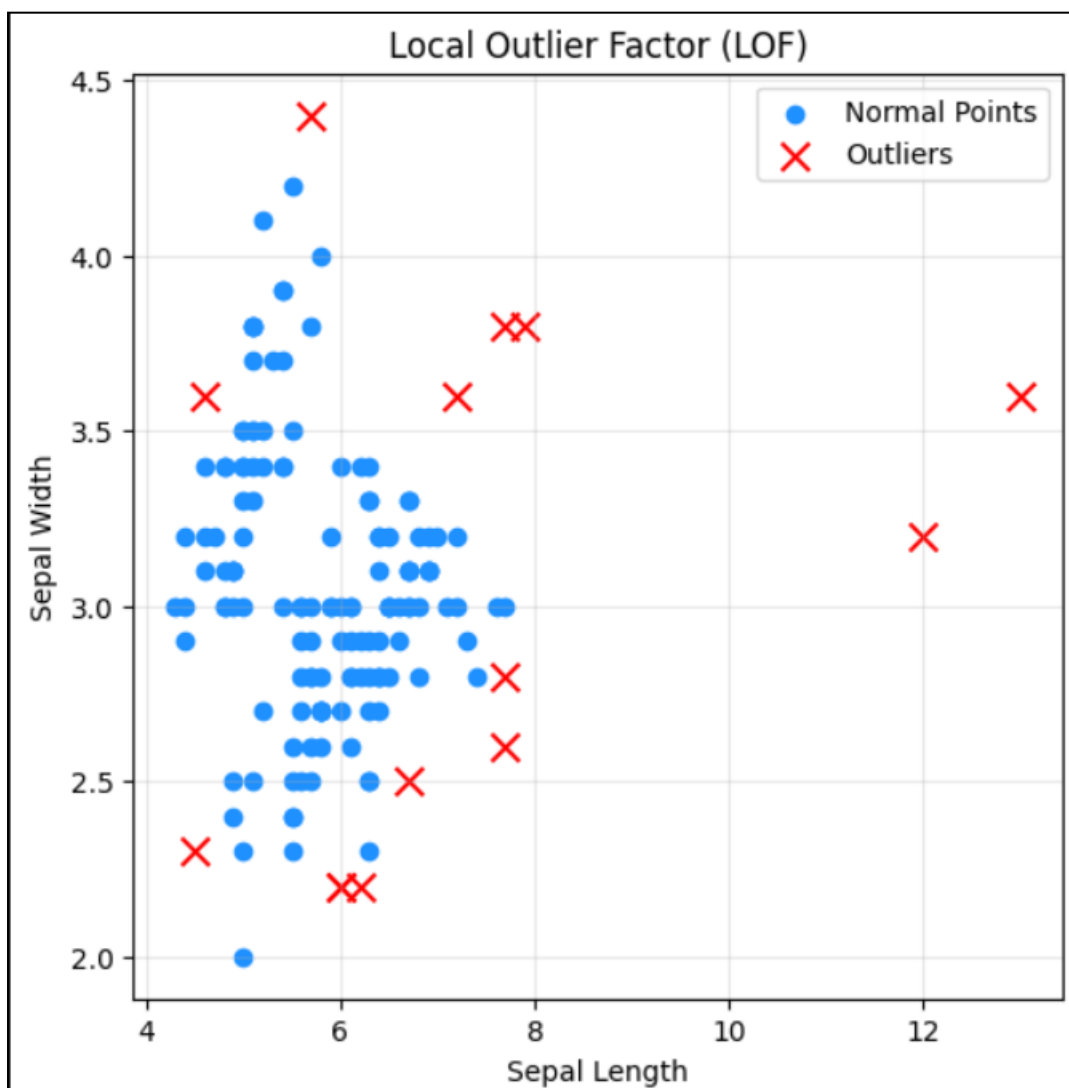
1. Write short note on Local Outlier Factor and Box Plot method of outlier detection.

A) Local Outlier Factor (LOF)

Theory:

Local Outlier Factor (LOF) is an unsupervised machine learning algorithm used to identify outliers by comparing the local density of a data point with the density of its neighboring points. Data points having significantly lower density than their neighbors are considered outliers.

Diagram (LOF):



Formula:

$LOF(p) = \text{Average Local Reachability Density of Neighbors} / \text{LRD of Point } p$

where,

- **p** = Data Point
- **LRD** = Local Reachability Density

Code:

```
from sklearn.neighbors import LocalOutlierFactor
```

```
feature = df1[["sepal_length"]]
```

```
lof = LocalOutlierFactor(
    n_neighbors=10
)
```

```
prediction = lof.fit_predict(feature)
df1["LOF Prediction"] = prediction
```

```
print(df1[["sepal_length", "LOF Prediction"]])
print("\nOutliers")
print(df1[df1["LOF Prediction"] == -1])
```

Output:

```
...      sepal_length  LOF Prediction
0         5.1           1
1         4.9           1
2        12.0          -1
3         4.6           1
4        13.0          -1
..      ...           ...
145        6.7           1
146        6.3           1
147        6.5           1
148        6.2           1
149        5.9           1

[150 rows x 2 columns]
```

Outliers

	sepal_length	sepal_width	petal_length	petal_width	species	\
2	12.0	3.2	1.3	0.2	setosa	
4	13.0	3.6	1.4	0.2	setosa	
13	4.3	3.0	1.1	0.1	setosa	

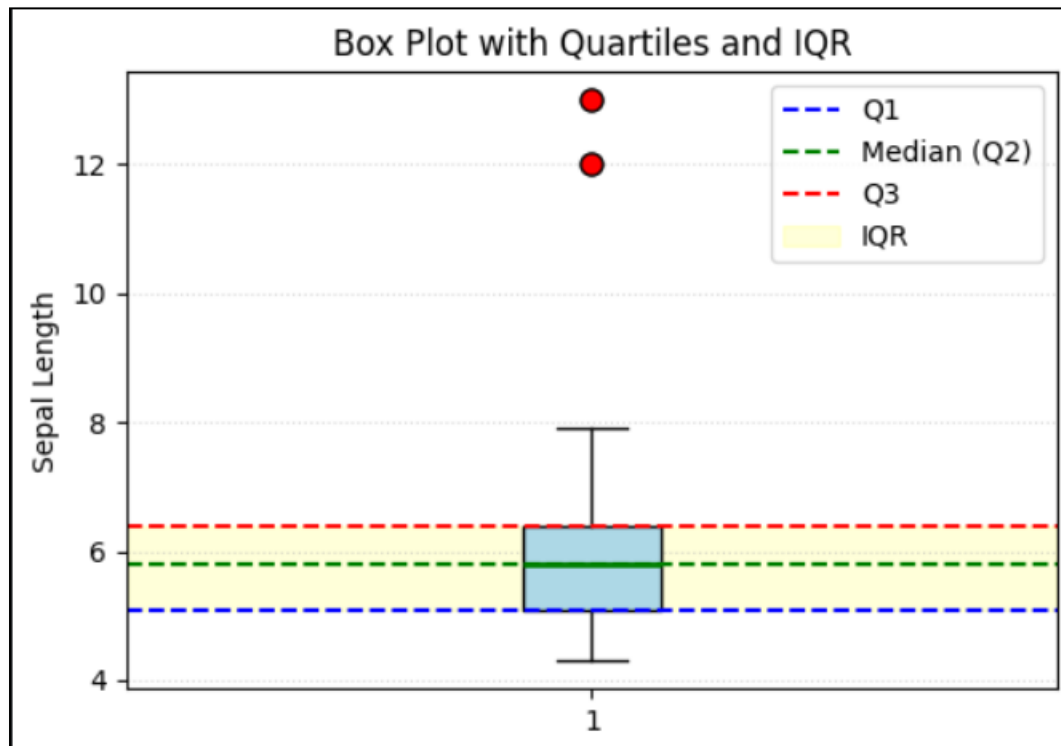
	LOF Prediction
2	-1
4	-1
13	-1

B) Local Outlier Factor (LOF)

Theory:

- The Box Plot Method is a graphical method used to detect outliers using the Interquartile Range (IQR).
- The box represents the middle 50% of the data.
- Any value below the Lower Bound or above the Upper Bound is considered an outlier.

Diagram (LOF):



Formula:

$$\text{IQR} = Q3 - Q1$$

$$\text{Lower Bound} = Q1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q3 + 1.5 \times \text{IQR}$$

Code:

```
# Select Feature  
feature = df1["sepal_length"]
```

```
# Calculate Quartiles  
Q1 = feature.quantile(0.25)  
Q2 = feature.quantile(0.50)  
Q3 = feature.quantile(0.75)
```



```
# Calculate Interquartile Range
```

```
IQR = Q3 - Q1
```

```
# Calculate Lower and Upper Bounds
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
# Detect Outliers
```

```
outliers = df1[(feature < lower) | (feature > upper)]
```

```
# Display Results
```

```
print("Q1 :", Q1)
```

```
print("Median (Q2) :", Q2)
```

```
print("Q3 :", Q3)
```

```
print("IQR :", IQR)
```

```
print("Lower Bound :", lower)
```

```
print("Upper Bound :", upper)
```

```
print("\nDetected Outliers\n")
```

```
print(outliers)
```

Output:

```
... Q1 : 5.1
    Median (Q2) : 5.8
    Q3 : 6.4
    IQR : 1.3000000000000007
    Lower Bound : 3.1499999999999986
    Upper Bound : 8.350000000000001

    Detected Outliers

      sepal_length  sepal_width  petal_length  petal_width  species  \
2             12.0           3.2           1.3           0.2  setosa
4             13.0           3.6           1.4           0.2  setosa

      LOF Prediction
2                -1
4                -1
```

2. A dataset is highly skewed. Which outlier detection method would you choose and why?

```
feature = skewedDataFrame["Value"]

Q1 = feature.quantile(0.25)
Q3 = feature.quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

iqrOutliers = skewedDataFrame[
    (feature < lower) |
    (feature > upper)
]
print("Outliers Detected using IQR")
print(iqrOutliers)
```

```
... Outliers Detected using IQR
      Value
11    7.007115
34    6.741261
50    6.985614
69    8.668293
100   15.000000
101   18.000000
102   20.000000
```

```
from scipy.stats import zscore

skewedDataFrame["Z-Score"] = zscore(
    skewedDataFrame["Value"]
)

zScoreOutliers = skewedDataFrame[
    np.abs(skewedDataFrame["Z-Score"]) > 3
]

print("Outliers Detected using Z-Score")
print(zScoreOutliers)
```

```
... Outliers Detected using Z-Score
      Value  Z-Score
100    15.0   3.928120
101    18.0   4.855348
102    20.0   5.473501
```

Conclusion:

- The dataset is positively skewed.
 - The IQR Method successfully detects the extreme values without assuming a normal distribution.
 - The Z-Score Method relies on the mean and standard deviation, making it less effective for skewed data.
 - Therefore, the IQR Method is more suitable for detecting outliers in highly skewed datasets.
-

3. A dataset has 40% missing values in one column. What strategy would you adopt and why?

If a column contains 40% missing values, the strategy depends on the importance of that column.

- If the column is not important for the analysis, it is better to drop the column using `dropna(axis=1)` to avoid introducing inaccurate information.
- If the column is important, the missing values should be filled using `fillna()` with an appropriate value such as the mean, median, or mode, depending on the type and distribution of the data.

Reason:

- Dropping an unimportant column simplifies the dataset and removes incomplete information.
- Filling missing values preserves useful data and avoids losing a large number of records.
- The choice between dropping and filling should be based on the significance of the column and its impact on the analysis.

4. Download a dataset containing categorical data. Standardize a categorical column:

```
# Load dataset
df = pd.read_csv(os.path.join(path, "netflix_titles.csv"))

# Select the categorical column
categoryData = df.copy()

# Introduce some inconsistencies
categoryData.loc[0, "type"] = " Movie "
categoryData.loc[1, "type"] = "movie"
categoryData.loc[2, "type"] = "MOVIE"
categoryData.loc[3, "type"] = "Tv Show"
categoryData.loc[4, "type"] = "TV SHW"

print(categoryData["type"].head())
```

```
*** 0      Movie
    1      movie
    2      MOVIE
    3    Tv Show
    4    TV SHW
    Name: type, dtype: object
```

• Convert to lowercase

```
categoryData["type"] = categoryData["type"].str.lower()

print("After Converting to Lowercase\n")

print(categoryData["type"].head())
```

```
*** After Converting to Lowercase

    0      movie
    1      movie
    2      movie
    3    tv show
    4    tv shw
    Name: type, dtype: object
```

• Remove extra spaces

```
categoryData["type"] = categoryData["type"].str.strip()

print("After Removing Extra Spaces\n")

print(categoryData["type"].head())
```

... After Removing Extra Spaces

```
0      movie
1      movie
2      movie
3  tv show
4  tv shw
Name: type, dtype: object
```

• Correct spelling errors

```
categoryData["type"] = categoryData["type"].replace({
    "tv shw": "tv show"
})

print("After Correcting Spelling Errors\n")

print(categoryData["type"].head())
```

... After Correcting Spelling Errors

```
0      movie
1      movie
2      movie
3  tv show
4  tv show
Name: type, dtype: object
```